# CAS, когерентность кешей и горячий мьютекс

От рантайма Go до микроархитектуры процессора

- Как четыре байта sync.Mutex.state превращаются в межъядерную точку сериализации.
- CAS → владение кеш-линией → MESI → барьеры памяти → ложное разделение → конкуренция под нагрузкой.

**Цель: связать одну строку Go-кода с физической передачей права владения между ядрами.**

# Одна строка Go — несколько аппаратных слоев



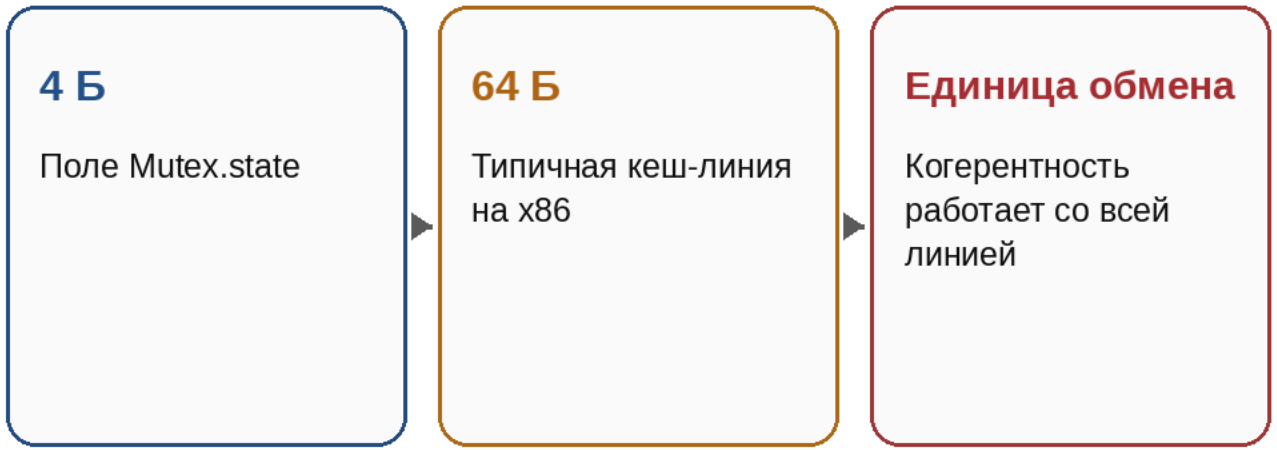

**Общее изменяемое состояние имеет не только логическую, но и физическую цену.**

# Процессор работает через иерархию памяти

- Регистры и кеши L1/L2/L3 находятся ближе к ядру, чем DRAM.
- Чтение из кеша обычно обслуживается быстрее, чем обращение к оперативной памяти.
- Запись в память не означает немедленного физического обновления DRAM.

**Для конкурентности важны видимость и порядок изменений, а не только местонахождение байтов.**

# Кеш хранит линию, а не отдельный int32




**Отсюда ложное разделение: независимые поля могут конфликтовать через одну кеш-линию.**

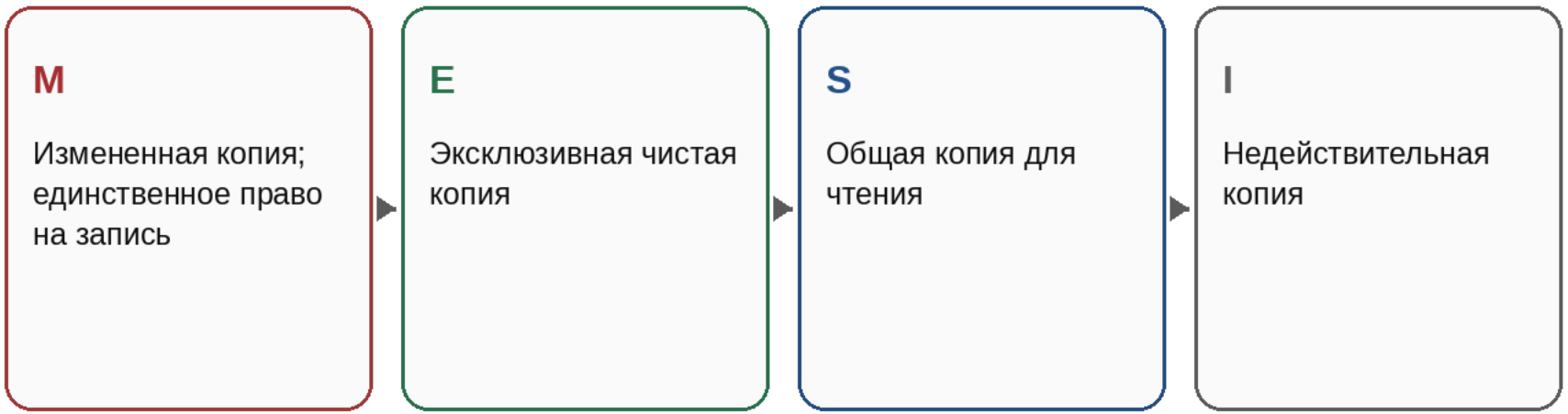
# Одна кеш-линия может находиться в нескольких L1




Чистые копии могут одновременно находиться в кешах разных ядер.

Для записи одному ядру потребуется эксклюзивное право владения линией.

# MESI как учебная модель




**Современные CPU могут использовать MESIF/MOESI и другие состояния. MESI здесь — учебная модель.**

# Совместное чтение и запись стоят по-разному

## Совместное чтение

- Ядра 0 и 1 могут одновременно иметь копию линии в состоянии S.
- Оба читают без передачи права на запись.

## Запись

- Пишущее ядро должно сделать остальные копии недействительными.
- Линия становится эксклюзивной или измененной у одного ядра.

# S → M: что требуется перед записью



# CAS — атомарная операция «сравнить и заменить»

```
if *ptr == old {
    *ptr = new
    return true
}
return false
```

- Чтение, сравнение и условная запись образуют одну атомарную операцию.
- Два ядра не могут одновременно успешно захватить один и тот же Mutex.state.

# CAS в Go на amd64

```
// концептуально
LOCK CMPXCHG [state], new
```

- Префикс LOCK обеспечивает атомарную операцию с согласованием между ядрами.
- Обычно не требуется блокировать всю шину: работает протокол когерентности кешей.

# CAS в Go на ARM64

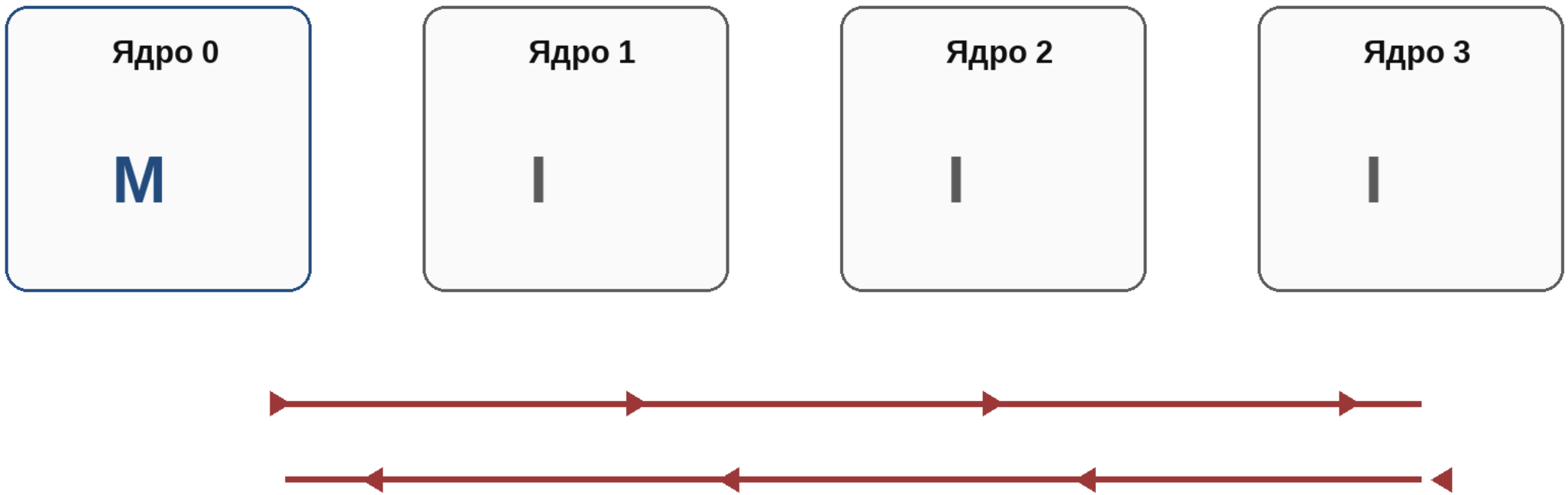
```
// путь LSE
CASAL ...

// резервный путь
LDAXRW ...
STLXRW ...
// повторить при неудаче
```

- Набор инструкций иной, но атомарность и гарантии Go должны сохраняться.
- При конкуренции за одну кеш-линию все равно нужно согласование между ядрами.

# Почему горячий Mutex.state «прыгает» между ядрами

Одна кеш-линия с правом на запись — один текущий владелец




**Право на изменение линии постоянно переходит между ядрами**

# Пинг-понг кеш-линии

- «Пинг-понг» кеш-линии — многократная передача права на изменение между ядрами.
- Межъядерный обмен может проходить через LLC, каталоги и межсоединение.
- Следующая запись ждет согласования владения, даже если сама операция короткая.

**Не обязательно копировать всю линию L1 → L1; обязательна смена права на запись.**

# Горячий Mutex.state — точка сериализации

- Все конкурирующие горутины модифицируют один атомарный int32.
- Атомарность требует согласованной последовательности смены права владения кеш-линией.
- Количество горутин может расти, но одна линия не станет многопоточной.

# Почему дополнительные ядра иногда не ускоряют

- Каждое дополнительное ядро добавляет конкурента за одну общую кеш-линию.
- Новый CAS не устраняет передачу права на изменение линии.
- Пропускная способность достигает плато, а при сильной конкуренции может снижаться.

**Масштабирование общего состояния ограничено не только планировщиком, но и когерентностью кешей.**

# Стоимость CAS зависит от расположения кеш-линии

### Линия уже у этого ядра

- Право на запись уже принадлежит ядру.
- Межъядерное согласование минимально.
- CAS без конкуренции может быть относительно дешевым.

### Линия у другого ядра или общая

- Нужно запросить право на запись.
- Другие копии могут стать недействительными.
- Задержка выше и зависит от топологии.

# Неудачный CAS тоже создает нагрузку

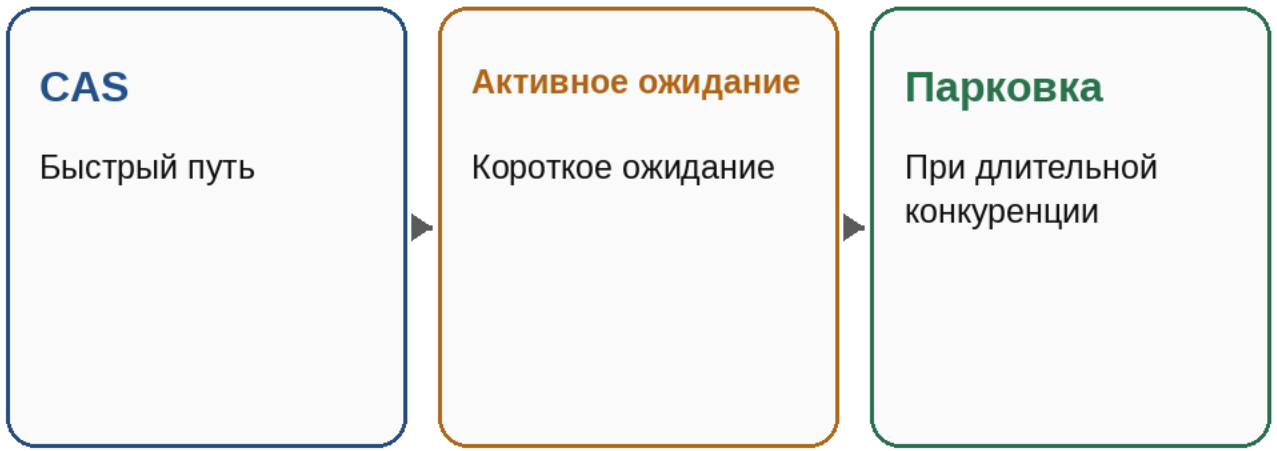
- Проигравший участник не захватил мьютекс, но его CAS все равно участвует в протоколе когерентности.
- Цикл повторных CAS поддерживает конкуренцию за одну горячую линию.
- Неблокирующий алгоритм не означает отсутствия аппаратной конкуренции.

# Почему бесконечный цикл CAS вреден

```
for !state.CompareAndSwap(0, 1) {
// повторить попытку
}
```

- Все активные ядра непрерывно обращаются к одному адресу.
- CPU занят, хотя полезная работа может не продвигаться.
- Кеш-линия постоянно переходит между ядрами.

# Зачем Mutex паркует ожидающие горутины




**Парковка снижает загрузку CPU и число активных претендентов на горячую кеш-линию.**

# Когерентность кешей и порядок памяти — разные задачи

## Когерентность кешей

- Согласует копии одной кеш-линии между ядрами.
- Определяет, кто может изменять линию.
- Реализуется протоколами семейства MESI/MOESI.

## Порядок операций с памятью

- Определяет, в каком порядке записи становятся наблюдаемыми для других ядер.
- Компилятор и CPU могут менять порядок операций.
- Нужны acquire/release и барьеры памяти.

# Проблема порядка операций с памятью

```
// Ядро 0
data = 42
ready = true

// Ядро 1
if ready {
use(data)
}
```

- Без синхронизации чтение ready == true не гарантирует корректного наблюдения data == 42.
- Модель памяти Go задает отношение «произошло до» (happens-before) для синхронизированных операций.

# Почему CPU меняет порядок операций

- Внеочередное исполнение помогает скрывать задержки зависимостей.
- Буферы записи позволяют продолжать работу до глобальной видимости записей.
- Спекуляция чтений и конвейер повышают пропускную способность.
- Примитивам синхронизации нужны дополнительные гарантии порядка.

# Барьер памяти (fence)




**Барьер памяти не «сбрасывает весь кеш в RAM». Он ограничивает порядок и видимость операций.**

# Семантика release/acquire

## Release — публикация

- Публикует предшествующие изменения перед освобождением синхронизации.
- Типичный Mutex.Unlock() использует семантику release.

## Acquire — получение

- После успешного захвата становятся доступны опубликованные изменения.
- Типичный Mutex.Lock() использует семантику acquire.

# Mutex публикует данные критической секции

```
// G1
mu.Lock()
user.Name = "Alice"
user.Age = 42
mu.Unlock()

// G2
mu.Lock()
use(user.Name, user.Age)
mu.Unlock()
```

- Mutex.state служит точкой синхронизации, а упорядоченность относится ко всем защищенным данным.
- Mutex.Unlock() синхронизируется с последующим соответствующим Mutex.Lock().

# Go скрывает большую часть деталей порядка памяти

- В C/C++ разработчик выбирает порядок операций: relaxed, acquire, release или seq_cst.
- Публичные атомики Go обеспечивают последовательную согласованность.
- Меньше тонких настроек — меньше ошибок; цена — меньше способов ослабить порядок вручную.

# Ложное разделение кеш-линии

Логически разные переменные могут физически находиться в одной кеш-линии




**Когерентность видит одну линию: запись a делает копию с b недействительной, и наоборот.**

# Истинное и ложное совместное использование

## Истинное совместное использование

- Несколько ядер реально изменяют одну переменную.
- Например общий счетчик или Mutex.state.
- Конфликт одновременно логический и физический.

## Ложное разделение

- Ядра изменяют разные поля.
- Поля лежат в одной кеш-линии.
- Когерентность все равно согласует право записи целой линии.

# Почему шардирование помогает аппаратно

- Один глобальный мьютекс — одна горячая кеш-линия.
- 16 или 32 шарда дают несколько независимых точек синхронизации.
- Вероятность одновременного запроса владения одной линией снижается.
- Сложность O(1) может сохраниться, а пропускная способность заметно вырасти.

# Production: глобальный мьютекс кеша

## До

- 50–100 тыс. запросов/с.
- Все обработчики: Lock → map lookup → Unlock.
- Поиск быстрый, но одно состояние мьютекса постоянно передается между ядрами.

## После шардирования

- Ключ распределяется между шардами.
- Несколько мьютексов и независимых кеш-линий.
- Меньше конкуренции и пинг-понга кеш-линий.

# Production: горячий атомарный счетчик

- Один atomic.Uint64 при миллионах обновлений/с может превратиться в горячую кеш-линию.
- Счетчики на воркер или на P уменьшают частоту записи в общее состояние.
- Периодическая агрегация снижает конкуренцию ценой дополнительной сложности и задержки учета.

# NUMA: стоимость зависит от топологии

- Передача владения между близкими ядрами обычно дешевле, чем межсокетный обмен.
- Глобальный мьютекс на 128-ядерном сервере может вести себя по-разному на разных NUMA-узлах.
- Задержка атомиков зависит от топологии, размещения потоков и кеш-линий.

# Как проверить конкуренцию нагрузочным тестом

```
type Counter struct {
    mu sync.Mutex
    n  uint64
}

func (c *Counter) Inc() {
    c.mu.Lock()
    c.n++
    c.mu.Unlock()
}

// go test -bench=. -cpu=1,2,4,8,16
```

- Пропускная способность перестает расти при увеличении числа CPU.
- После этого сравниваем ту же операцию с шардированием.
- На Linux можно дополнить анализ perf stat / perf c2c.

# Диагностика по слоям

- pprof: профиль мьютексов и блокировок показывает конкуренцию за локи в Go.
- go tool trace: видно ожидание, пробуждение и задержки планировщика.
- Нагрузочный тест: воспроизводимо сравниваем масштабирование при разном числе CPU.
- perf / PMU: при необходимости измеряем cache-to-cache обмен и аппаратные события.

# Полная причинно-следственная цепочка



# Что важно запомнить

- CAS — атомарная операция чтения-модификации-записи; стоимость зависит от состояния кеш-линии.
- Когерентность кешей и порядок операций с памятью решают разные задачи.
- Горячая запись вызывает передачи владения и физическую сериализацию.
- Парковка уменьшает активную конкуренцию за кеш-линию и загрузку CPU.
- Ложное разделение создает конкуренцию без логически общей переменной.
- Шардирование распределяет записи по независимым кеш-линиям.

**Общее изменяемое состояние имеет физическую цену в межъядерной подсистеме.**

# Источники для углубления

- Модель памяти Go — go.dev/ref/mem.
- Документация sync и sync/atomic — pkg.go.dev/sync, pkg.go.dev/sync/atomic.
- Реализация атомарных операций Go — internal/runtime/atomic.
- Intel 64 and IA-32 Architectures Software Developer’s Manual.
- Arm Architecture Reference Manual — разделы об атомиках и порядке памяти.